



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

F.CSM101 Программчлалын үндсэн аргууд

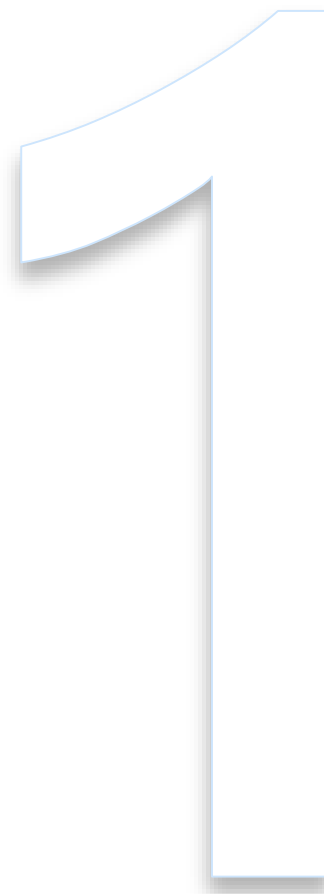
Логик программчлалын парадигм

Лекц 11

*Доктор (Ph.D) **Ганбаатарын ГАНБАТ***

2023-2024 Хавар

- Дедүкц тооцоолол
 - Жишээ
 - Үр ашиг
- Синтакс
 - Цагаан толгой
 - Нөхцөл (Term)
 - Томьёо (Formula)
- Логик программ
- Нэгтгэлийн онол (Theory of Unification)



R.Kowalski: Программчлалын идэвхтэй байдал (activity) бол

Алгоритм = Логик + Хяналт

- Шийдлийн логик : *What compute?*
- Хяналт тал: *How of find solution?*

- Программчлалын *декларатив* хэв маяг
- *Логик дедукц* дээр суурилсан
- PL-ний логик: *Теоремын автомат баталгаа* хийх санаанд суурилсан

1970, 1980s AI программчлалын хувьд алдартай болсон (мэдлэгийн декларатав илэрхийлэл. Гэхдээ AI өөр зам сонгосон.)

- Зорилго: **Жагсаалт эрэмбэлэлтийн** алгоритм зохиох
- Императив PL
 - Жагсаалтын элементүүдийг хэрхэн дахин байрлуулахыг алхам алхмаар тайлбарлана
- Логик PL
 - **Байгуулж болох баталгаа** удирдах:
 - Жагсаалт бүрийн хувьд ижил элементүүдээс бүрдсэн эрэмбэлэгдсэн жагсаалт байдаг

Уламжлалт императив хэл дээр хамтад нь явч явдаг. **Логик программчлал (LP)**-д программист зөвхөн логик шаардлагыг өгөх бөгөөд, хяналт талыг хийсвэр машинд хариуцуулдаг.

Дедукц тооцоолол (Deduction as Computation)

- Тооцоолол бол логик дедукц гэх үзэл (1930s, К.Годель, Ж.Хербранд)
 - LP-ын хэлний тооцоолох суурь механизм гэж хардаг **нэгтгэх (unification)** процессын тухай зарим санаа (Хербранд).
 - Анхны санааг өгсөн формал тодорхойлолт (1960s, А.Робинсон).
 - Уг тохиромжтой автомат формал дедукцыг тооцооллын механизм болгон интерпретац хийж болохыг ойлгосон (1970s, эхээр)
- Prolog (**PRO**gramming in **LOG**ic) хэлний олон хувилбар + хэд хэдэн хэл.
 - Үр ашгийн үүднээс хяналтыг ил заадаг бүтэцтэй. Логик интерпретацыг шууд хийхгүй. *(Энд хэлний семантиктай зөрчилдөж, логик парадигмын цэвэр декларатив байдлыг бууруулдаг)*
 - Гэхдээ л LP хэл гэж үзнэ. Программист асуудлын зааварыг арай чанартай томъёолох (зарлах) болдог.
 - Зарим тохиолдолд программ нь гайхалтай товч, энгийн, ойлгомжтой байна.

$i \in [1, 9]$, хоёр i -ийн хооронд яг i ширхэг тоо байх, i бүр 3 удаа орсон жагсаалт ол.

- Зөв "...1, 2, 1, 8, 2, 4, 6, 2..."
- Буруу "...1, 2, 1, 8, 2, 4, 2, 6..."

- "Юу" хийх нь ойлгомжтой ч, "хэрхэн" хийх нь тодорхой биш
- **Шийдлийн жишээ**, маш энгийн (мөн үр ашиггүй):
 - i бүр 3 тул 27 урттай бүх сэлгэмлийг байгуулж, шаардлага хангахыг нь олох.
 - Декларатив зарчмаар, **унар *npenukam (predicate)*** list_of_27(Ls):

```
list_of_27(Ls):-
    Ls = [_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_].
```

- Энд “_” бол **нэргүй (anonymous)** хувьсагч,
- Дэд жагсаалт: [1 , x , 1 , y , 1] , нэргүй хувьсагч хэрэглэвэл: [1 , _ , 1 , _ , 1]
- X нь Y-ын дэд жагсаалт байх **бинар препикат** sublist(X, Y) *байн гэж үзвэл:*

```
sublist ([1,_,1,_,1],  Ls)
```

[illegible]

- “,” –ийг “AND” гэж үзвэл логик агуулгаараа конъюнкц (2-11).
- Процедур нь интерпретацгүй бол PL-д хөрвүүлэх шаардлагатай, ашиггүй
- Интерпретораар үнэлүүлэн шийдээ олох боломжтой Prolog программ

- 1-р мөр, `sol` процедурын зарлалт `Ls` гэсэн нэг формал параметртэй.
 - Процедурын их бие нь 2-11 мөрийн дагуу арван процедурын дуудалттай.
- 2-р мөр, `list_of_27` процедурыг бодит `Ls` параметртэй дуудна.
 - `list_of_27` нь бодит параметрийг нэргүй 27 хувьсагчийн жагсаалт болгоно.
- 3-11 мөрүүд адилхан дуудалттай. Жнь:

```
sublist([9,_,_,_,_,_,_,_,_,_,9,_,_,_,_,_,_,_,_,_,_,9], Ls)
```

- `Ls` хувьсагчийн үүссэнээр эхний параметр нь хоёр дахь параметрт (`Ls`)-ын дэд жагсаалт байх жагсаалт бий болно гэж үзнэ.
- Прологт, процедур дуудалт нь текст дээрх эрэмбэтэй (цэвэр логик программд бол эрэмбэ заахгүй).
- `sol(Ls)` дуудалт нь асуудлын шийдлээр үүсгэгдсэн `Ls`-ийг буцаана, Жнь:
`1, 9, 1, 2, 1, 8, 2, 4, 6, 2, 7, 9, 4, 5, 8, 6, 3, 4, 7, 5, 3, 9, 6, 8, 3, 5, 7.`

- Өмнөх процедур интерпретацаар комплекс асуудлыг шийдэж программыг авсаархан, харьцангуй энгийн илэрхийлэх боломжийг олгодог. Энэ боломжийн оронд хэл нь үр ашгийн зардалтай байдаг.
- Өмнөх программ энгийн боловч гүйцэтгэх тооцоолол нь комплекс
 - Интерпретатор нь өгөгдсөн бүх нөхцлийг хангасныг олох хүртлээ дэд жагсаалтын янз бүрийн хослолыг туршиж үзэх ёстой.
 - Энэ хайлтын процесст **буцаах (backtracking)** механизмыг ашигладаг.
 - Тооцооллыг үргэлжлүүлэх боломжгүй бол энэ хүртэл хийсэн тооцооллыг цуцалж, хэрэв дахиад өөр дэд жагсаалт сонгон шалгана (хэрэв өөр хувилбар байхгүй бол тооцоо амжилтгүй дуусна).
 - Ерөнхийдөө хайлтын процесс нь экспоненциал хүндрэлтэй.

- Логик программ нь тохиромжтой логик **томъёо** (*formulae*)-ны олонлог.
 - 1-р эрэмбэ (first-order) логик, **предикат тооцоолол** (*predicate calculus*)
 - Тухайн D домайны элементийн шинжийг симболоор илэрхийлдэг
- Өндөр эрэмбийн (high-order) логик
 - Илүү хүнд аргументтэй предикатыг зөвшөөрдөг
 - D дээрх олонлог болон функц (2-р эрэмбэ), тэдгээрийн олонлог (3-р эрэмбэ) гэх мэт D-ийн нэмэлт элементүүд
- **1-р эрэмбийн логик хэл (The Language of First-Order Logic)**
 1. **Цагаан толгой** (*alphabet*).
 2. Цагаан толгойн дээр тодорхойлсон **нөхцөл** (*terms*).
 3. Цагаан толгойн дээрх **зөв** (*well-formed*) **томъёо** (*formulae*).

- **Логик** симбол олонлог. ***logical symbols***:
 - $\wedge$ (конъюнкц), $\vee$ (дизъюнкц), $\neg$ (үгүйсгэл), $\rightarrow$ (импликац), $\leftrightarrow$ (давхар $\rightarrow$)
 - Хэллэгийн тогтмолууд *true, false*.
 - $\exists$ (оршин байх) ба $\forall$ (дурын) кванторууд.
 - "(" ба ")" хаалт, таслал "," зэрэг тусгаарлах тэмдэг.
 - $X, Y, Z, \dots$ хувьсагчийн хязгааргүй V олонлог (тоолох боломжтой)
- **Нон-логик (логик өргөтгөл/extra-logical)** симбол олонлог
 - *предикат бүхий тэмдэглэгээ (signature with predicate)* (Σ, Π) .
 - **Функц Σ** : аргументын тоо (arity)-той функц симбол (arity 0 бол тогтмо/ constants $a, b, c, \dots$; 1-ээс их бол $f, g, h, \dots$)
 - **Предикат Π** : аргументын тоотой предикат симбол: p, q, r
 - Σ ба Π нь хоосон огтлолцолтой, логик симбол олонлогоос тусдаа гэж үзье.
 - Нэг нь функц, нөгөө нь харьцаа (предикат) гэж тайлбарлагдана.

- Функцууд нь оролт ба гаралтыг хооронд нь ялгадаг
 - Императив/функциональ PL-д: Үр дүнг үүсгэхийн тулд аргументуудад функцийг хэрэглэнэ
- Предикатууд нь оролт, гаралтыг ялгадаггүй
 - Логик PL-д: Предикат үнэн байх утгуудыг хайх

- Математик логик, CS-ны үндэс. Далд хэлбэрээр оршдог. Жнь:
 - Арифметик илэрхийлэл нь операндуудад операторуудыг хэрэглэх нөхцөл.
 - Strings, Three, List г.м бүтцийг тохирох байгуулагч ашиглах нөхцөл гэж үзнэ.
 - Семантик утгаараа бол D домайны элементийг илэрхийлнэ
- Синтаксийн хувьд аргументын тоон шаардлагыг хангах. Жнь:
 - хэрэв a, b тогтмол; X, Y хувьсагч; f, g нь 2 arity-тай: $f(a, b)$ болон $g(a, X)$ г.м
 - Нөхцлийг аргумент болгож болно. $g(f(a, b), Y)$ эсвэл $g(f(a, f(X, Y)), X)$ г.м

Тод 12.1 (Нөхцөл) Σ тэмдэглээ (мөн хувьсагчийн олонлог V) дээрх нөхцлийг индуктив байдлаар тодорхойлно:

- Хувьсагч (V дээрх) нь нөхцөл.
- Хэрэв f (Σ дээрх) нь n аргумент тоотой функц тэмдэглэгээ, $t_1, \dots, t_n$ нь нөхцөл бол $f(t_1, \dots, t_n)$ нь нөхцөл.

- Хэлний зөв томьёогоор нөхцлийн чанарыг (property) илэрхийлнэ.
 - Семантикийн хувьд томьёо нь D домэйнэ элемент шинжийг илэрхийлнэ. Жнь, $>(3, 2)$ нь "3" нөхцөлтэй утгаас нь "2" нөхцөлтэй утга их.
 - Логик симбол $\rightarrow$ комплекс томьёо. Жнь, $>(X, Y) \wedge >(Y, Z) \rightarrow >(X, Z)$.
- Шууд предикат дээрх хэрэглээгээр бүтээгдсэн бол **Атомик томьёо**.

Тод 12.2 (Томьёо) (Σ, Π) нөхцөлтэй тэмдэглэгээ дээрх (зөв) томьёог индуктив байдлаар тодорхойлно:

- Хэрэв $t_1, \dots, t_n$ нь Σ тэмдэглэгээ дээрх нөхцөл, $p \in \Pi$ нь n тооны аргументтай предикат симбол бол $p(t_1, \dots, t_n)$ нь томьёо болно.
- True, false нь томьёо.
- Хэрэв F ба G нь томьёо бол $\neg F, (F \wedge G), (F \vee G), (F \rightarrow G), (F \leftrightarrow G)$ нь томьёо.
- Хэрэв F нь томьёо, X нь хувьсагч бол $\forall X. F, \exists X. F$ нь томьёо.

Time break: Prolog example

```
has_exam(X) :- is_course(X), gives_grade(X) .
```

```
is_course(pp) .
```

```
gives_grade(pp) .
```

```
?- has_exam(pp) .
```

Time break: Prolog example

Means "implication"



```
has_exam(X) :- is_course(X) , gives_grade(X) .
```

```
is_course(pp) .
```

```
gives_grade(pp) .
```



Means "and"

```
?- has_exam(pp) .
```


Time break: Prolog example

Means "implication"



`has_exam(X) :- is_course(X), gives_grade(X) .`

`is_course(pp) .`



`gives_grade(pp) .`

Means "and"

`?- has_exam(pp) .`



Evaluates to "true"

Логик программ

- Логик програмчлал нь теоремын автомат баталгааны боловсруулалтыг илүү үр дүнтэй хийх томъёоны тусгай классуудтай.
 - Жнь *resolution* гэдэг гаргалгааны тусгай дүрэмтэй *клаус (clause)*.
- *Бодит (definite) клаус, SLD resolution* гэсэн хоёр ойлголтыг ашиглан баталгааг хайх процедур нь маш энгийн болдог бөгөөд
 - *Бодит (definite) клаус* (Клаус ойлголтын хязгаарлагдмал хэлбэр)
 - *SLD resolution* (resolution-ы хязгаарлагдмал хэлбэр)
 - Баталгаанд шаардлагатай хувьсагчдыг утгыг ил тооцоолох боломж олгодог

тод 12.3 (*Логик программ*): $H, A_1, \dots, A_n$ атомик томъёо байг. Тодорхойлох клаус нь дараах хэлбэрийн томъёо юм:

$$H : \neg A_1, \dots, A_n.$$

Программ нь үнэн гэж үзсэн клаус олонлогийн хүрээнд ажилладаг

- Ерөнхий хэлбэр (Prolog): `<term> :- <term>`
 - Хоёр тал өгөгдсөн бол: **Дүрэм (Rule)**
 - Зөвхөн зүүн тал өгөгдсөн бол : **Факт (Fact)**
 - Зөвхөн баруун тал өгөгдсөн бол : **Зорилго** (эсвэл query)
 - Ихэвчлэн `--`-ийн оронд `?-` гэж бичдэг.
- Нөхцөл (нэр томьёо) нь дараах хэлбэртэй (Prolog):
 - **Тогтмол**: Атом (жижиг үсгээр эхлэх ёстой), Тоо эсвэл тэмдэгт мөр
 - **Хувьсагч** (том үсгээр эхлэх ёстой)
 - **Бүтэц**: Маягтын логик предикат
 - `<functor>(<arg1>, ... <argN>)`

- Асуултанд хэрхэн хариулах вэ (өөрөөр хэлбэл зорилгодоо хүрэх)?
- Хоёр гол санаа
 - **Resolution**: Мэдэгдэж байгаа клаусуудад үндэслэн нөхцлийг солих
 - **Unification**: Хоёр нөхцөл ижил байгаа эсэхийг тогтооход "Паттерн тааруулах"
- Resolution зарчим
 - C_1 ба C_2 клаусууд өгөгдсөн
 - Хэрэв C_1 -ийн толгой нь C_2 -ын бие дэх t -тэй тохирч байвал:
 - t -г C_1 -ийн их биеээр сольж болно

Time break: Prolog example

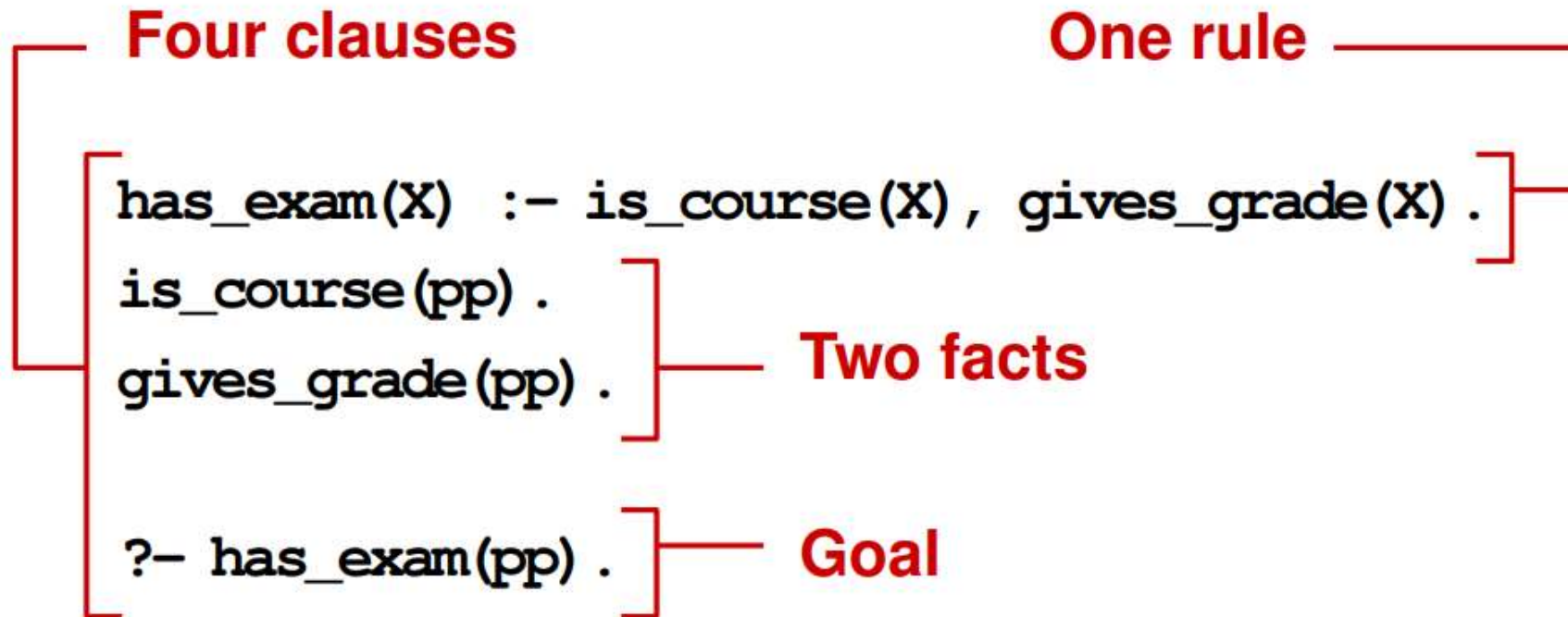
```
has_exam(X) :- is_course(X), gives_grade(X) .
```

```
is_course(pp) .
```

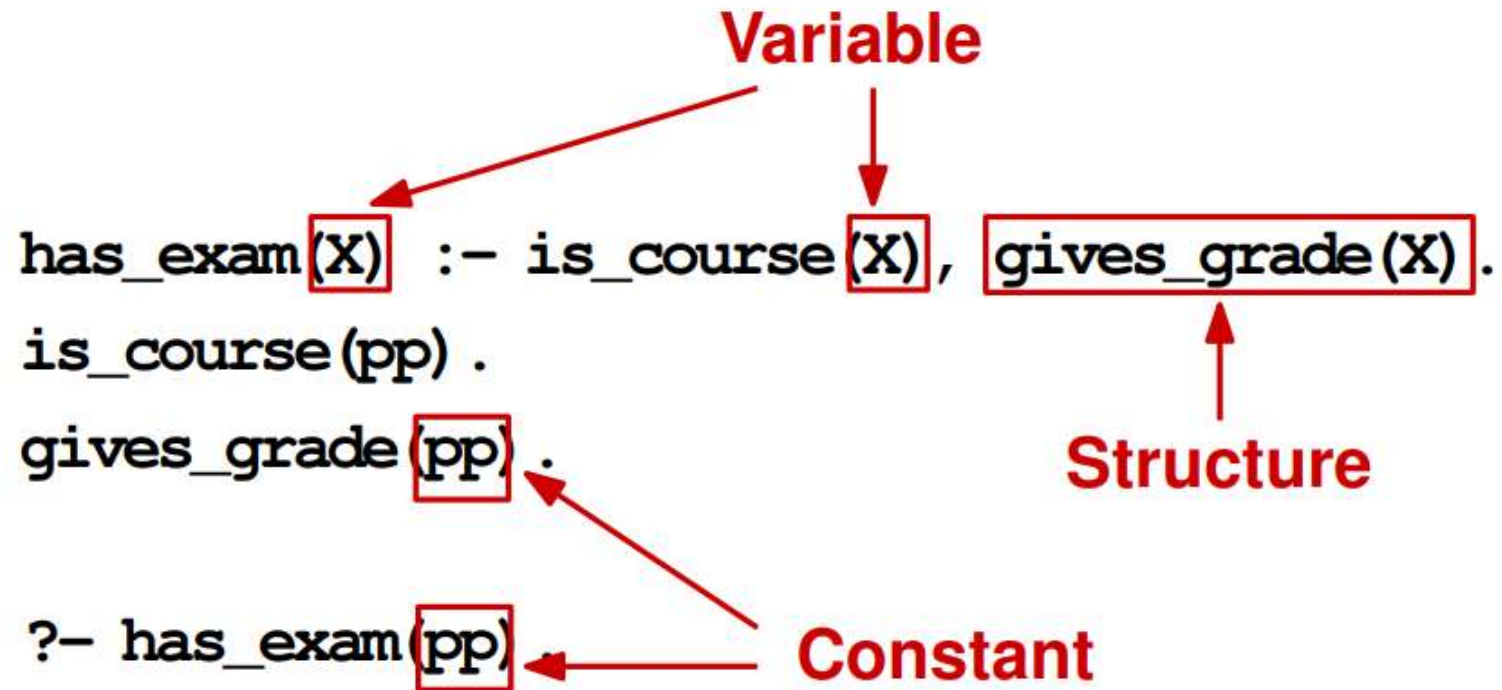
```
gives_grade(pp) .
```

```
?- has_exam(pp) .
```

Time break: Prolog example



Time break: Prolog example



Нэгтгэлийн онол (Theory of Unification)

- LP-ын тооцооллын суурь механизм нь нэгтгэх (unification) процедурыг хэрэглэн нөхцөл хоорондын тэгшитгэлийг шийддэг.
 - Хувьсагч болон нөхцлийг холбох орлуулгыг тооцоолдог.
- UNIFICATION ДҮРЭМ
 - Тогтмол нь зөвхөн өөртэйгөө л нэгддэг
 - Хоёр бүтэц нэгдмэл байх тохиолдол:
 - Ижил функц
 - Адил аргументын тоо
 - Аргументууд рекурсив байдлаар нэгддэг
 - Хувьсагч нь бусадтайгаа нэгддэг
 - .. утгатай: Хувьсагчийг үүсгэсэн
 - .. өөр хувьсагчтай: Хоёулаа ижил утгыг авна

- Predetermined олонлогоос утгыг авч болох үл мэдэгдэх *логик хувьсагч*
- *LP-ын логик хувьсагч VS императив хэлний modifiable хувьсагч*
 1. Логик хувьсагчийг ганц л удаа холбож болно. (Императив, утга оноох).
 2. Логик хувьсагчийн утгыг хэсэгчлэн/шатлан тодорхойлж болно (эсвэл undefined). Императив хэлний механизмаас өөр.
 3. Логик хувьсагчийн холболт хоёр чиглэлтэй. Хувьсагчид холбосон нөхцлийг өөрчилсноор утгыг нь засахаас гадна хувьсагчид өөр холболт өгч нөхцлийг нь өөрчилж болно
 - Хэрэв X нь $f(Y)$ нөхцөлтэй холбогсон, дараа нь X -г $f(a)$ нөхцөлд холбож чаддаг бол Y хувьсагчийг a тогтмолтой холбоно.
- Сүүлийнх нь логик програмыг огт өөр аргаар хэрэглэг боломжийг олгодог болохоор суурь зарчим болдог.

- Хувьсагч ба нөхцөл хоорондын холбоо нь орлуулах ойлголтын үүднээс хийгдсэн байдаг. Ихэвчлэн ϑ , σ , ρ , ... грек үсгээр

тод 12.4 (Орлуулга): Орлуулга нь хувьсагчаас өөрт нь байдаггүй төгсгөлөг тооны хувьсагчтай нөхцөл руу чиглэсэн функц. ϑ орлуулгыг дараах хэлбэрээр тэмдэглэнэ.

$$\vartheta = \{X_1/t_1, \dots, X_n/t_n\}$$

Энд $X_1, \dots, X_n$ нь өөр хувьсагчид, $t_1, \dots, t_n$ нь нөхцлүүд бөгөөд $i = 1, \dots, n$ байхад t_i нь X_i -ээс ялгаатай гэж үздэг.

Тооцооллын загвар

- "Дедункц тооцоолол" гэсэн санааг хэрэгжүүлдэг логик парадигм нь өмнө үзсэн бүх загвараас эрс ялгаатай тооцооллын загвартай.
 1. Цэвэр логикт загварт байж болох цорын ганц утга нь өгөгдсөн тэмдэглээн дээрх нөхцөл байна.
 2. Программ нь бүхэлдээ логиктой декларатив уншилттай эсвэл үйлдлийн хэв маягийн процедур уншилттай байж болно.
 3. Тооцоолол нь нөхцөл (тиймээс зорилгод) дээр гарч ирж буй хувьсагчдыг нэгтгэх механизмаар бусад нөхцөл руу оруулах замаар ажилладаг.
 4. Хяналт нь бүхэлдээ автоматаар буцаах (backtracking) процесс дээр суурилсан хийсвэр машинаар удирдуулдаг (Prolog-д байдаг зарим боломжит тэмдэглэгээг эс тооцвол).



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

АНХААРАЛ ТАВЬСАНД БАЯРЛАЛАА